

Operating System, Section 2

Assignment1 - Simple MyShell Document

유준혁

32212808

Department of Mobile Systems Engineering

2025-09-20

Table of Contents

1. 서론
2. 소프트웨어 요구사항 기술 (Requirements)
3. 소프트웨어 설계 (Design)
4. 소프트웨어 구현 (Implementation)
5. 시험 & 디버깅 (Verification)
6. 소프트웨어 유지보수 (Maintenance)
7. 결론

1. 서론

A. 개발 목적

본 프로젝트의 목적은 `fork()`, `execve()`, `wait()` 등의 시스템 콜을 사용하여 간단한 셸을 구현하고, 이를 통해 운영체제의 핵심 기능인 프로세스의 생성과 실행 과정을 이해하는 것이다.

2. 소프트웨어 요구사항 기술 (Requirements)

A. 기능적 요구사항

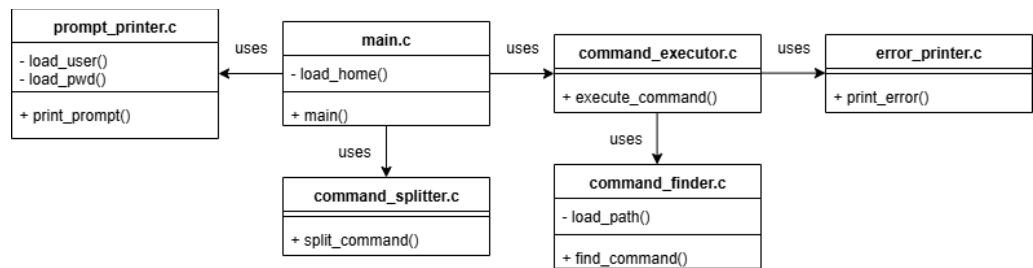
Makefile 작성	셸 프로그램을 make 명령어로 컴파일할 수 있어야 한다.
셸 시작	실행 파일을 실행하면 셸 프로그램이 시작되어야 한다.
셸 종료	<code>exit</code> 또는 <code>quit</code> 를 입력하면 셸 프로그램이 종료되어야 한다.
프롬프트 변경	환경 변수 (<code>PWD</code> , <code>USER</code>)를 <code>getenv()</code> 를 통해 불러오고, 프롬프트에 출력되어야 한다.
명령어 입력	명령어를 입력 받을 수 있어야 한다.
인자 전달	인자를 입력 받고, <code>execve()</code> 를 통해 전달할 수 있어야 한다.
경로 탐색	경로 절대 경로가 아닐 경우, 환경 변수 <code>PATH</code> 의 디렉토리들을 탐색해야 한다.
프로그램 실행	자식 프로세스를 <code>fork()</code> 를 통해 생성하고, 명령어를 실행할 수 있어야 한다.
실행 대기	부모 프로세스는 <code>wait()</code> 를 통해 자식 프로세스가 종료될 때까지 대기해야 한다.
반복 실행	자식 프로세스가 종료된 후에도 셸 프로그램은 종료되지 않아야 한다.

B. 비기능적 요구사항

에러 처리	입력이 잘못된 경우, 오류 메시지를 출력해야 한다.
	실행 파일이 없는 경우, 오류 메시지를 출력해야 한다.
	권한이 없는 경우, 오류 메시지를 출력해야 한다.
POSIX	POSIX 시스템 콜 사용을 준수해야 한다.
성능	사용자의 입력에 대한 응답이 즉각적이어야 한다.
유지보수성	모듈화 된 구조를 가져야 한다.

3. 소프트웨어 설계 (Design)

A. 아키텍처 설계



4. 소프트웨어 구현 (Implementation)

A. 개발 환경 & 도구

Operating System	Ubuntu 22.04.5 LTS
Kernel	Linux 5.15.0-136-generic
Integrated Development Environment & Editor	vi
Programming language	C
Framework	stdlib.h: Standard library stdio.h: Standard input/output string.h: String library sys/wait.h: Process wait handling unistd.h: POSIX API
Build Tool	GCC compiler with Makefile

B. 모듈별 기능 설명

main.c	사용자의 입력 대기, 내장 명령어 (exit, quit, cd)처리, wait()를 통한 자식 프로세스 종료 대기를 수행한다.
prompt_printer.c	환경 변수 (PWD, USER)를 프롬프트에 출력한다.
command_splitter.c	사용자의 입력을 명령어와 인자로 분리한다.
command_executor.c	fork()를 통한 자식 프로세스 생성, execve()를 통한 명령어 실행을 수행한다.
command_finder.c	경로가 절대 경로가 아닐 경우, 환경 변수 PATH의 디렉토리를 탐색한다.
error_printer.c	오류가 발생한 경우, 적절한 오류 메시지를 출력한다.

C. 주요 알고리즘 설명 & Code Snippet

```
if(strcmp(argv[0], "cd") == 0){
    if(argv[1] == NULL){
        chdir(load_home());
    }else if(chdir(argv[1]) != 0){
        print_error("Failed to change directory");
    }
    continue;
}
```

사용자가 내장 명령어 cd 를 입력한 경우, 위와 같이 main.c 에서 디렉토리를 변경한다. argv[1]이 없는 경우, 홈 디렉토리로 이동하고, 실행을 실패한 경우 에러 메시지를 출력한다. cd 와 같은 내장 명령어는 fork()를 필요로 하지 않는다.

```
// Split command
split_command(input, argv);

// External command
pid_t pid = fork();
if(pid < 0){
    print_error("Failed to fork");
}else if(pid == 0){
    execute_command(argv);

    // If execution fail, exit
    exit(EXIT_FAILURE);
}else{
    wait(NULL);
}
```

사용자가 ls -l 등의 외장 명령어 입력한 경우, 위와 같이 main.c 에서 split_command()가 실행한 후, fork()를 통해 자식 프로세스를 생성하고, execute_command()를 실행한다. 부모 프로세스는 wait()를 통해 자식 프로세스가 종료될 때까지 대기한다.

```
// Split command
int split_command(char *input, char **argv){
    int argc = 0;

    // Split string
    for(char *token = strtok(input, " "); token != NULL; token
= strtok(NULL, " ")){
        argv[argc++] = token;
    }

    // Save NULL at the end of array argv to call execve
    argv[argc] = NULL;
    return argc;
}
```

위와 같이 split_command()는 사용자의 입력을 공백을 기준으로 나누어 배열 argv 에 저장하고, 배열 argv 의 끝에 NULL 을 추가하여 반환한다.

```
// Execute command
void execute_command(char **argv){
    // Convert to absolute path
    char *path = find_command(argv[0]);

    // If command is not found, print error and exit
    if(!path){
        print_error("Command not found");
        exit(EXIT_FAILURE);
    }

    // Execute command
    execve(path, argv, NULL);

    // If execution fails, print error and exit
    print_error("Failed to execute command");
    exit(EXIT_FAILURE);
}
```

위와 같이 execute_command()는 find_command()를 실행한 후, execve()를 통해 명령어를 실행한다. 명령어 실행에 실패한 경우, 에러 메시지를 출력하고 자식 프로세스를 종료한다.

```

// Find command
char* find_command(char *cmd){
    // If command starts with '/' or '.', return as-is
    if(cmd[0] == '/' || cmd[0] == '.'){
        return cmd;
    }

    // Copy environment variable PATH
    char path_copy[1024];
    strncpy(path_copy, load_path(), sizeof(path_copy));
    path_copy[sizeof(path_copy)-1] = '\0';

    // Tokenize environment variable PATH
    char *token = strtok(path_copy, ":");

    // Find command
    static char path[1024];
    while(token){
        snprintf(path, sizeof(path), "%s/%s", token, cmd);
        // If command is executable, return path
        if(access(path, X_OK) == 0){
            return path;
        }

        // Try next directory
        token = strtok(NULL, ":");
    }

    // If command is not found, return NULL
    return NULL;
}

```

위와 같이 find_command()는 환경 변수 PATH를 탐색하여 실행 가능한 명령어를 찾는다. 명령어가 '/', '.'로 시작하는 경우, 그대로 반환하고, 명령어를 찾지 못한 경우 NULL을 반환한다.

5. 시험 & 디버깅 (Verification)

시험 목록	입력	출력	상태
Makefile 빌드	make	.o 파일 생성	Pass
Makefile 클리어	make clear	.o 파일 삭제	Pass
셸 프로그램 실행	./myshell	셸 프로그램 실행	Pass
셸 프로그램 종료	exit	셸 프로그램 종료	Pass
	quit	셸 프로그램 종료	Pass
프롬프트 확인	없음	환경 변수 (PWD, USER)출력	Pass
인자 전달	echo Hello	Hello 출력	Pass
	ls -l /home/	home 디렉토리 출력	Pass
상대 경로로 실행	ls	현재 디렉토리 출력	Pass
절대 경로로 실행	ls /home/	home 디렉토리 출력	Pass
반복 실행	pwd → date → whoami	각 명령어의 결과 출력	Pass
잘못된 명령어	hello	Command not found 출력	Pass
실행 파일 없음	cat hello.c	No such file or directory 출력	Pass

6. 소프트웨어 유지보수 (Maintenance)

본 프로젝트는 유지보수와 확장성을 고려하여 설계되었으며, 구체적으로 다음과 같은 사항을 고려하였다.

모듈화 설계 & 확장성	기능을 각각의 모듈로 분리하였다. 각 모듈이 단일 책임 원칙 (SRP, Single Responsibility)을 준수하기 때문에 코드 수정 시 다른 모듈에 대한 영향을 최소화했다. 예를 들어, prompt_printer.c 모듈을 수정하는 것만으로도 프롬프트와 프롬프트에 출력되는 환경 변수를 수정할 수 있다.
에러 처리	error_printer.c 모듈을 통해 모든 오류 처리를 일원화하여, 유지보수 시 에러 메시지 관리가 용이하도록 설계하였다.
코드 가독성	모든 모듈과 함수에 대한 주석을 작성하여 코드 이해도를 높였다.

7. 결론

fork(), execve(), wait() 등의 시스템 콜을 사용하여 간단한 셸을 구현하고, 이를 통해 운영체제의 핵심 기능인 프로세스의 생성과 실행 과정을 이해하였다. 그리고 환경 변수를 사용할 때, 그대로 사용하는 것이 아니라 복사 후 사용해야 한다는 점도 알게 되었다. 예를 들어, 환경 변수 PATH 를 그대로 사용하면 시스템 전체에 영향을 줄 수 있다. 문자열을 복사하고 작업하는 것은 까다로웠지만 좋은 학습이 되었다.

```
// External command
pid_t pid = fork();
if(pid < 0){
    print_error("Failed to fork");
}else if(pid == 0){
    execute_command(argv);

    // If execution fail, exit
    exit(EXIT_FAILURE);
}else{
    wait(NULL);
}

// Execute command
void execute_command(char **argv){
    // Convert to absolute path
    char *path = find_command(argv[0]);

    // If command is not found, print error and exit
    if(!path){
        print_error("Command not found");
        exit(EXIT_FAILURE);
    }

    // Execute command
    execve(path, argv, NULL);

    // If execution fails, print error and exit
    print_error("Failed to execute command");
    exit(EXIT_FAILURE);
}
```

또한 방어적 코딩이라는 개념을 알게 되었다. 예를 들어, 위와 같이 command_executor.c 에서 exit()를 호출하고 있지만, main 에서도 exit()를 호출하고 있다. 처음에는 중복 코딩은 나쁜 것이라고 생각하였지만, 예외 상황을 이중으로 방지하여 프로그램의 안전성을 확보하는 것도 중요하다는 것을 알게 되었다.

마지막으로 완전한 모듈화를 구현하고 싶었지만, 아키텍처 설계 과정에서 내장 명령어를 고려하지 못했기 때문에, 내장 명령어는 main.c 에 하드코딩 되었다. 추후에 내장 명령어도 별도의 모듈로 분리하면 완전한 모듈화에 더욱 가까운 프로그램이 될 수 있을 것이라고 생각한다.